

High-Performance DBMSs with io_uring: When and How to Use It

Matthias Jasny
Technische Universität Darmstadt
matthias.jasny@tu-darmstadt.de

Muhammad El-Hindi
Technische Universität München
muhammad.el-hindi@tum.de

Tobias Ziegler
TigerBeetle
tobias@tigerbeetle.com

Viktor Leis
Technische Universität München
viktor.leis@tum.de

Carsten Binnig
Technische Universität Darmstadt &
DFKI & hessian.AI
carsten.binnig@tu-darmstadt.de

ABSTRACT

We study how modern database systems can leverage the Linux io_uring interface for efficient, low-overhead I/O. io_uring is an asynchronous system call batching interface that unifies storage and network operations, addressing limitations of existing Linux I/O interfaces. However, naively replacing traditional I/O interfaces with io_uring does not necessarily yield performance benefits. To demonstrate when io_uring delivers the greatest benefits and how to use it effectively in modern database systems, we evaluate it in two use cases: Integrating io_uring into a storage-bound buffer manager and using it for high-throughput data shuffling in network-bound analytical workloads. We further analyze how advanced io_uring features, such as registered buffers and passthrough I/O, affect end-to-end performance. Our study shows when low-level optimizations translate into tangible system-wide gains and how architectural choices influence these benefits. Building on these insights, we derive practical guidelines for designing I/O-intensive systems using io_uring and validate their effectiveness in a case study of PostgreSQL’s recent io_uring integration, where applying our guidelines yields a performance improvement of 14%.

PVLDB Reference Format:

Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-Performance DBMSs with io_uring: When and How to Use It. PVLDB, 19(9): 2317-2330, 2026.
doi:10.14778/3819518.3819553

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/mjasny/vldb26-iouring>.

1 INTRODUCTION

Modern hardware and the I/O bottleneck. Modern PCIe 5.0 hardware, including NVMe SSDs such as the Kioxia CM7-R (2.45M IOPS) and NICs such as the ConnectX-7 (400 Gbit/s), sustains millions of IOPS and hundreds of gigabits per second of throughput, yet conventional I/O interfaces struggle to saturate them [23, 24, 34, 41, 45]. In particular, kernel-based I/O interfaces, as still used by

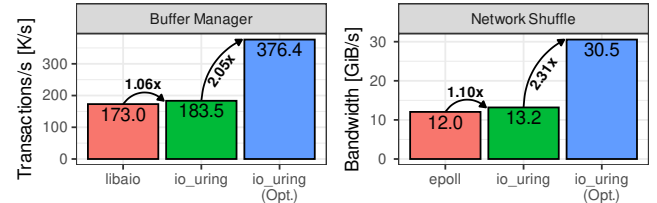


Figure 1: Performance comparison between traditional I/O interfaces and io_uring in a buffer manager and network shuffle. Naive use only yields modest gains, whereas designs that fully exploit io_uring more than double the performance.

production-grade database systems, incur system-call and context-switch overhead, consuming a significant fraction of CPU cycles without saturating these devices [46]. These inefficiencies widen the hardware–software gap and make low-overhead I/O mechanisms central to modern database systems.

Challenges of user-space I/O. User-space I/O frameworks such as DPDK, SPDK, and RDMA bypass the kernel and can deliver high performance on dedicated hardware [27, 35, 46]. However, operating entirely in user space removes OS abstractions such as file systems and TCP networking, making integration difficult for production databases that rely on them. These stacks also require exclusive control of SSDs or NICs [11, 18, 30], which may conflict with deployments where devices must be shared. Consequently, user-space I/O, despite its advantages, has not seen wide adoption and is used mainly in specialized, tightly controlled environments rather than general-purpose systems [11, 22].

io_uring features for efficient I/O. The Linux io_uring interface [4] is a promising candidate to bridge the gap between efficient I/O and the preservation of common kernel abstractions. It combines three key features, distinguishing it from earlier kernel I/O interfaces. First, *a unified interface* integrates storage, network, and other system calls into one framework. Second, *fully asynchronous execution* overcomes limitations of existing interfaces, allowing applications to perform useful work while I/O operations complete in the background. Third, *batched submission and completion* process multiple operations with a single system call, amortizing system call overhead and context switches. These features make io_uring attractive for database systems that issue large numbers of storage and network I/O operations.

Low-overhead I/O with io_uring? However, io_uring is not a panacea. Simply replacing traditional I/O interfaces with io_uring does not necessarily yield substantial performance benefits. As

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 9 ISSN 2150-8097.
doi:10.14778/3819518.3819553

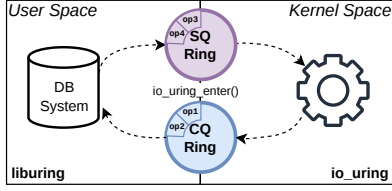


Figure 2: **io_uring architecture.** The database system in user space communicates with the kernel via two shared ring buffers: the Submission Queue (SQ) for enqueueing I/O requests and the Completion Queue (CQ) for receiving results.

Figure 1 shows, using `io_uring` off the shelf instead of `libaio` for storage I/O in a buffer manager, or instead of `epoll` for a network shuffle, only modestly improves performance (by 1.06× and 1.10×, respectively). In contrast, when the system is explicitly designed around `io_uring`’s capabilities (e.g., batching) and uses appropriate optimizations (e.g., registered buffers), the end-to-end performance improvements become much more pronounced: 2.05× for the buffer manager and 2.31× for the network shuffle.

These observations motivate our three research questions to guide DBMS system builders in using `io_uring`:

- (1) *When to use io_uring?* Under which conditions – especially I/O-intensive scenarios – does it provide the greatest benefit?
- (2) *How to integrate io_uring?* How should a DBMS architecture incorporate `io_uring` to exploit its capabilities effectively?
- (3) *How to tune io_uring?* Which `io_uring` features most strongly influence DBMS performance?

Contributions and outline. In this paper, we study `io_uring` as a kernel-based path to high-performance I/O that preserves standard OS abstractions in database systems, across both storage- and network-bound workloads. We evaluate `io_uring` using two complementary use cases supplemented with microbenchmarks. First, we integrate `io_uring` into a transactional storage engine on NVMe SSDs to examine storage-bound workloads (Section 3). Second, we employ `io_uring` for data shuffling in a distributed analytics engine on 400 Gbit/s networks, representing network-bound workloads (Section 4). From these case studies, we derive general principles for effective `io_uring` use and validate them by improving PostgreSQL’s `io_uring` backend to achieve more than 10% additional speedup (Section 5).

2 BACKGROUND ON IO_URING

`io_uring` was introduced into the Linux kernel in 2019 and has since been actively developed and optimized. Despite this progress, there has been little work on understanding how to adapt it to the requirements of database systems. This section therefore provides the background needed to understand how `io_uring` can be used in DBMSs and how its design influences performance. We highlight two aspects that distinguish `io_uring` from existing I/O backends: its application interface and its internal execution model, both of which enable high performance for data-intensive systems.

2.1 Interface of io_uring

Through its three key capabilities (unified I/O, asynchronous execution, and batching), `io_uring` provides an interface that aligns

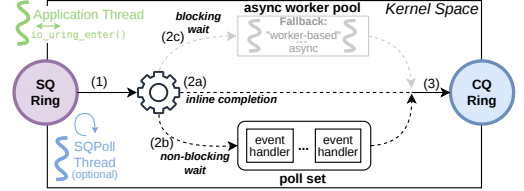


Figure 3: **io_uring provides three execution paths:** inline completion (2a), asynchronous execution via the poll set (2b), and a fallback to worker threads for blocking operations (2c). An optional SQPoll thread can submit requests without syscalls.

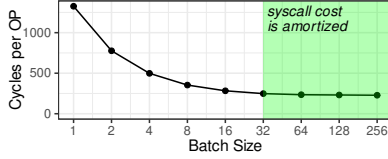
well with the demands of high-speed DBMSs. Below, we describe these capabilities in more detail and their implications for DBMSs.

Unification of I/O with io_uring. Traditional DBMSs rely on synchronous system calls such as `read()` and `write()`, which provide a simple, uniform abstraction but scale poorly due to their blocking behavior. To enable non-blocking network I/O, Linux introduced `epoll` to monitor multiple sockets for readiness. For storage, `libaio` provided a separate API that in practice was mostly restricted to direct block I/O and often lacked true asynchronous execution. This fragmentation forced developers to combine `epoll` and `libaio`, leading to duplicated code paths and limited concurrency between storage and network operations. `io_uring` eliminates the need for such hybrid designs by unifying storage and network (as well as other system calls) under a single fully asynchronous API. Its capabilities continue to expand beyond I/O-related system calls (e.g., `madvise`), moving toward a general-purpose asynchronous execution model for Linux. The unified interface enables DBMSs to overlap network and disk I/O more efficiently within a single path, simplifying system design and reducing context switching.

Asynchronous architecture. At a high level, `io_uring` is as a unified asynchronous layer over existing kernel I/O subsystems, such as the block layer for storage devices and the TCP/IP stack for networking. It is implemented in the kernel (Figure 2, right) and accessed through the `liburing` user-space library [9]. Unlike `epoll`’s readiness-based polling approach, `io_uring` employs a *completion-based* model, notifying applications after operations complete rather than when they become possible. It implements this model using two memory-mapped ring buffers: the *Submission Queue (SQ)* and the *Completion Queue (CQ)*. These queues are shared between user space and the kernel, avoiding additional data copies when submitting and completing requests. Applications enqueue I/O requests in the SQ, and their corresponding completions later appear in the CQ. Because completions may arrive out of order, each request carries a user-defined identifier to match submissions and completions. `io_uring` further supports *request linking* to enforce operation ordering for multiple elements in the SQ.

Batch processing. While `epoll` can report multiple readiness events, each I/O operation, such as `read()`, still requires its own syscall. In contrast, `io_uring` enables applications to enqueue multiple I/O requests in the SQ before triggering their submission with a single `io_uring_enter` syscall. Similarly, multiple completions can be retrieved from the CQ in one step. This batching capability amortizes syscall overhead and reduces context switches, allowing the kernel to process operations in bulk. Even modest batch sizes

(e.g., 16 operations) reduce the CPU cycles per operation by roughly 5–6× compared to single-operation submission as shown below:



As we will show in our use case discussions, DBMSs often have opportunities to issue I/O requests in batches, for example, during buffer-pool eviction or group commits [32, 39, 40]. However, excessive batching can also introduce drawbacks; therefore, it must be tuned carefully to yield clear benefits.

2.2 Inner Workings of io_uring

To effectively tailor DBMSs to io_uring, it is important to understand how I/O requests are executed. In the following, we examine the internals of io_uring and discuss other important aspects.

Issuing I/O requests. io_uring supports two ways to issue I/O in the kernel (step (1) in Figure 3). In the default mode, the application thread (top left) calls `io_uring_enter` and transitions into kernel mode, where it processes submissions and completions. The syscall can either block until a specified number of completions are available or return immediately. In contrast, when applications set up io_uring with the *SQPoll* mode, they avoid user–kernel transitions (syscalls) on the submission path by decoupling submission from execution. A dedicated kernel thread (cf. Figure 3, bottom left) continuously polls the SQ, issues I/O on behalf of the application, and posts results to the CQ. When no new requests are submitted, the SQPoll thread enters a sleep state after a configurable timeout. In a dedicated microbenchmark, we measured that waking this thread introduces a non-trivial latency of roughly 30 microseconds. As our study later demonstrates, choosing the most beneficial execution strategy requires a thorough understanding of the application architecture and workload characteristics.

Execution paths in io_uring. I/O in io_uring can follow three main paths as shown in Figure 3 (steps 2a–c):

(2a) *Inline execution.* When processing submissions, io_uring first attempts to complete requests inline, for example, when reading from a socket that has data already available. Such operations execute immediately and their completion is posted to the CQ.

(2b) *Non-blocking execution.* If an operation cannot be completed inline, its handling depends on its type. For pollable operations, such as non-blocking socket reads, io_uring installs an internal event handler (`io_async_wake()`) that is executed when the socket becomes readable (Figure 3, 2b). By default, io_uring waits indefinitely for the operation unless a timeout via `OP_LINK_TIMEOUT` is set.

(2c) *Blocking execution.* Certain operations cannot be executed asynchronously; for instance, blocking filesystem calls, such as `fsync`, or large storage reads. In such cases, io_uring delegates execution to worker threads (`io_worker`). This fallback is slower and incurs higher overhead than the native asynchronous paths. Applications can explicitly request this behavior using the `IOSQE_ASYNC` flag, which forces execution in a worker thread. In a dedicated microbenchmark, issuing NOPs that were handled by `io_worker` threads added an average overhead of 7.3 microseconds compared

to inline execution. This additional cost results from offloading to a separate thread and synchronization between the worker and submission context. Frequent fallback or a large number of active `io_worker` threads typically indicates suboptimal I/O patterns and may warrant application-level redesign [5–7].

Avoiding preemptions for completion-handling. When an asynchronous operation finishes, io_uring must run *task_work* in the kernel to place the completion entry into the CQ (step 3 in Figure 3). By default, this *task_work* runs whenever the application transitions from user to kernel space. If the thread is busy (for example, during a join or scan), the kernel may issue an inter-processor interrupt (IPI) to process pending completions. This effectively preempts the application, disrupts cache locality, increases jitter, and reduces batching efficiency. To mitigate these effects, io_uring offers the `COOP_TASKRUN` flag (CoopTR), which reduces IPIs and allows applications to delay *task_work*. However, completions are still processed on any kernel-user transition, including unrelated syscalls such as `mmap()`. Because preemptions have side effects, both the default and cooperative modes are ill-suited for modern high-performance DBMSs. The `DEFER_TASKRUN` flag (DeferTR) runs *task_work* only on `io_uring_enter` calls, making it the recommended mode [8] because it gives applications more control and eliminates unwanted preemptions. We therefore use it for the remainder of the paper, unless otherwise stated.

Other features for modern hardware. io_uring is designed to exploit modern hardware capabilities, supporting a variety of additional features. These span high-level application optimizations and low-level runtime tuning for asynchronous I/O. Key features include buffer registration and pinning to reduce memory management overhead, multishot operations, polling modes, and advanced request scheduling. In the remainder of this paper, we examine how DB engines leverage these mechanisms to implement efficient I/O.

3 EFFICIENT STORAGE I/O WITH IO_URING

Modern NVMe devices can sustain millions of IOPS, yet conventional I/O stacks rarely reach this potential [23, 27]. To explore if and how io_uring can close this gap, we discuss our three research questions (when to use, how to integrate, and how to tune io_uring) in the context of a buffer-managed storage engine.

We follow a stepwise approach during its design to highlight how io_uring’s key capabilities impact system performance. After our use case discussion, we use targeted microbenchmarks to isolate specific io_uring behaviors in the storage context and provide insights for estimating achievable I/O gains.

3.1 Use Case: Buffer-Managed Storage Engine

We use a buffer-managed storage engine as our primary use case because the buffer manager sits directly on the I/O path of out-of-memory transactions and orchestrates data movement between memory and SSDs. We outline its core responsibilities and architecture (Figure 4), and then discuss how to exploit io_uring effectively. **Buffer manager overview.** A buffer manager caches frequently accessed pages and loads or evicts them as needed. When a requested page is not present in the buffer pool, a *page fault* triggers a read I/O to retrieve it from storage. If the buffer pool is full, the buffer manager must select a page for eviction; if it is dirty, it is

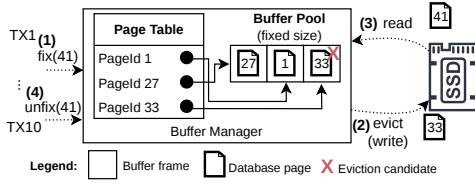


Figure 4: Overview of the buffer-managed storage engine design. Cold pages are evicted and written to disk freeing space that is used to cache frequently accessed pages.

written back before its buffer frame is reused. Because these operations lie on the critical transaction path, their efficiency is crucial for sustaining high throughput. Although background tasks, such as checkpointing, also interact with the buffer manager and issue additional I/O, we ignore them for simplicity.

Buffer manager architecture. The buffer manager maintains a preallocated pool of buffer frames and a page table mapping logical page identifiers to frames, along with metadata such as reference bits and dirty flags. It exposes two primitives [16]: `fix(page_id)` (1) checks whether the requested page resides in the buffer pool and loads it from storage (3) if not, evicting (2) another page if necessary, while `unfix(page_id)` (4) releases the page and marks it dirty if modified. When the pool is full, a replacement policy selects victims. In this paper, we use clock-sweep [20], a common algorithm in DBMS: pages are marked during the first pass and, if still unreferenced on the second pass, dirty pages are written back before the frame is reused. The storage engine includes a B-tree index for tuple access and updates. When the working set fits in memory, these operations complete without I/O; otherwise, page faults and evictions place I/O on the critical path, coupling buffer management with application logic.

3.2 Workload & System Conditions

As mentioned in Section 1, the question *when to use io_uring* – when `io_uring` provides measurable performance gains – depends on workload and system conditions. We therefore use simple back-of-the-envelope models based on I/O cost and CPU utilization to estimate expected gains and validate the resulting system behavior.

Experimental setup. Unless stated otherwise, our experiments run on single-socket servers with an AMD EPYC 9654P processor (96 cores, SMT disabled), 768 GiB of DRAM, and Linux kernel 6.15.0 on Ubuntu 24.04 LTS with IOMMU disabled. For storage experiments, one server is equipped with eight PCIe 5.0 NVMe SSDs (Kioxia CM7-R). For network experiments, each node is equipped with a PCIe 5.0 Nvidia ConnectX-7 NIC (400 Gbit/s Ethernet) and connected through a 400 Gbit switch.

Workload conditions. To capture the impact of different workload characteristics, we use two standard DBMS benchmarks: single-statement, I/O-intensive YCSB-like transactions and the more complex, compute-bound TPC-C workload. The experiments use a small 1 GB buffer pool. For YCSB, we load 10 million tuples (8-byte key, 128-byte value), which with index structures and metadata results in a roughly 70% page fault probability under uniform updates with 4 KiB pages, producing an I/O-bound workload well suited for storage analysis. For TPC-C, we use 1 and 100 warehouses to study the effects of a mostly in-memory vs. a mostly out-of-memory setting.

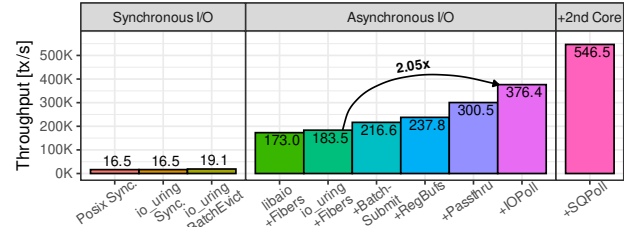


Figure 5: YCSB throughput (100% uniform updates, one update per transaction) under different buffer manager designs and I/O execution modes. `io_uring` features and design optimizations are enabled incrementally from left to right, increasing transaction throughput from 16.5 k to 546.5 k TPS.

System conditions. We evaluate several buffer manager configurations on one of these servers. The configurations range from fully synchronous to batched and asynchronous variants to study how the workloads interact with system conditions. All configurations utilize a single-threaded setup, in which one core handles transaction processing and I/O requests. This setup isolates I/O behavior from concurrency effects, allowing more precise analysis of performance improvements. Later, in Sections 4 and 5.2, we extend the analysis to multithreaded configurations. Our evaluation compares `io_uring` against conventional kernel-based I/O interfaces used in production systems. We do not include kernel-bypass frameworks such as SPDK as direct baselines, since they target a different design space and would require substantial changes to the storage stack.

3.3 Using io_uring in the Storage Engine

Traditional buffer managers perform I/O through blocking system calls such as `pread()` and `pwrite()`, where each I/O request must complete before the thread can proceed. As a performance baseline, we implement a synchronous buffer manager on top of `io_uring` by submitting one request at a time and waiting for its completion, ensuring that the DBMS thread has at most one outstanding I/O. Although this baseline uses `io_uring`'s submission and completion queues for consistency with later variants, it does not exploit its asynchronous or batching features.

When io_uring does not help. This setup is the simplest form of I/O execution, where the transaction thread blocks on every page fault or for eviction. Consequently, throughput is directly tied to device latency, and using `io_uring` does not result in performance gains. For the update-heavy YCSB workload mentioned earlier, our posix-based and `io_uring`-based implementation reaches a single-threaded throughput of 16.5 k tx/s (Figure 5, *Posix* and *io_uring*). Because in-memory updates are negligible compared to storage latency, both implementations are *I/O-latency bound*.

Modeling the bottleneck. To validate the performance results, we use a simple latency-based model derived from the operation costs in Table 1. The numbers in Table 1 are measured independently using targeted experiments: the transaction execution cost is obtained from an in-memory run, the I/O costs are measured separately for the corresponding read/write paths, and only the 70% page fault rate follows from the configured YCSB workload. Assuming a 70% page fault rate and an average read-plus-write latency of $70 + 12 = 82 \mu\text{s}$ for our SSD device, the expected throughput

is $\frac{1}{0.7 \times 82 \times 10^{-6}} \approx 17.4$ k tx/s. The estimate aligns with the measured 16.5 k tx/s, confirming that I/O latency rather than CPU or software overhead limits synchronous performance. As a consequence, higher throughput can be achieved by reducing effective I/O latency or amortizing the latency of one request through batching.

Table 1: I/O numbers used for performance modeling.

Single Read	Single Write	Transaction Execution	Single Read	Batch Read	Batch Write
70 μ s	12 μ s	8264 clk	10200 clk	5400 clk	5700 clk
I/O LATENCY		CPU CYCLES			

3.3.1 Using io_uring to Batch Writes

Synchronous I/O is bound by I/O latency, since each page fault triggers a blocking writeback of a dirty page before the next read can proceed. Although io_uring cannot reduce device latency, its *batching feature* can reduce the impact on the critical path.

Amortizing eviction cost. We leverage io_uring to introduce *batched write submission* for the buffer manager’s eviction path. Instead of evicting and writing one page at a time, the buffer manager collects multiple victims and issues their writes together with a single io_uring_enter() call. While execution remains synchronous and reads and writes do not yet overlap, batching lowers submission overhead and exploits device-level parallelism, demonstrating how a minor architectural change can benefit from io_uring’s strengths. **Performance implications.** Batching write operations improves performance by about 14%, reaching 19 k tx/s (compare Figure 5, +BatchEvict) because submission overhead for eviction is amortized. Eviction writes are issued in batches, so their latency is incurred once per batch rather than per eviction. This amortizes the cost across N evictions and leaves the 70 μ s read latency as the dominant term in the latency model from the previous section. The expected throughput is $\frac{1}{0.7 \times 70 \times 10^{-6}} \approx 20.4$ k tx/s. The measured and predicted results align closely, confirming that batching removes the write latency from the latency-bound path and effectively mitigates latency through amortization rather than elimination.

3.3.2 Using io_uring for Asynchronous I/O

While batched writes amortize write latency, the buffer manager still operates synchronously, blocking on page faults and leaving the CPU idle during I/O. Multiple threads, as studied in Section 3.6, hide this latency but introduce synchronization and scheduling overhead. To hide latency in our single-threaded design, we therefore adopt *asynchronous transaction execution* to overlap I/O and computation. **Overlapping compute & I/O.** io_uring’s completion-based model integrates naturally with async runtimes such as coroutines or fibers. We extend the buffer manager with fibers [2] for cooperative scheduling. Each transaction runs as a fiber that issues I/O requests and yields on page faults, allowing the io_uring-based runtime to schedule other fibers and keep the CPU active.

Cooperative transaction execution. Fiber context switches cost only tens of CPU cycles since they save and restore only register state, providing efficient user-level concurrency suited for I/O-intensive workloads. When a suspended fiber’s I/O completes, it is marked ready and resumed by the scheduler. Since all concurrency is cooperative, the B-tree implementation requires neither locks nor atomic operations across fibers. If a fiber resumes after an I/O delay and the data structure has changed, it restarts the B-tree traversal to ensure correctness and preserve isolation without explicit

synchronization. Considering such details is important for accurate performance modeling, as shown later.

CPU, the new bottleneck. With up to 128 fibers, throughput rises by nearly an order of magnitude to 183 k tx/s (Figure 5, +Fibers). Under the same asynchronous execution scheme, libaio reaches 173 k tx/s, so we continue the analysis on io_uring, which provides higher throughput and exposes additional optimization opportunities. At this concurrency level, the system becomes CPU- rather than latency-bound: concurrent fiber execution hides I/O latency and the CPU is fully utilized. We therefore switch from a latency- to a cycle-based model that accounts for per-transaction CPU cost.

Using hardware cycle counters (rdtsc), we measure transaction logic (B-tree traversal and update) as $c_{tx} = 8,264$ cycles, and I/O processing (submission and completion) as $c_{io} = c_{read-single} + c_{write-batch} = 15,900$ cycles (Table 1). For a 3.7 GHz core and a page fault rate of $r_{pf} = 70\%$, the expected throughput is $\frac{\text{clock frequency}}{c_{tx} + r_{pf} \times c_{io}} = \frac{3.7 \times 10^9}{8264 + 0.7 \times 15900} \approx 190.8$ k tx/s, matching the measured 183 k tx/s. This confirms that CPU overhead, rather than I/O latency, now dominates performance, as intended with asynchronous execution.

3.3.3 Using io_uring to Batch Reads

In the initial asynchronous design, each fiber submits its I/O request right before blocking and is woken up once the I/O completes. This hides I/O latency by overlapping reads and writes but incurs syscall overhead for each I/O. We therefore introduce *batched read submission*, which groups read requests from multiple fibers before entering the kernel via io_uring_enter(). The batched submission amortizes syscall overhead and exploits device-level parallelism, reducing per-I/O cycle cost (Table 1) and improving CPU efficiency. **Adaptive batching.** Read batching improves throughput by lowering per-I/O cost but may introduce queuing delays if the runtime waits too long to collect requests, while very small batches negate the amortization benefit. Our runtime therefore uses *adaptive batching*, adjusting the batch size based on the ratio of outstanding I/Os to waiting fibers. When many I/Os are in flight, additional submissions are deferred to increase amortization; when few are pending, batches are flushed earlier to keep the CPU busy. This feedback mechanism maintains high device utilization while avoiding stalls when the scheduler runs out of runnable fibers.

Impact of adaptive batching. Performance increases by about 18%, from 183 k to 216 k tx/s (Figure 5, +BatchSubmit). Adding I/O cost for batched reads to the model, $c_{io} = c_{read-batch} + c_{write-batch} = 11,100$ cycles (Table 1), yields $\frac{3.7 \times 10^9}{8264 + 0.7 \times 11100} \approx 230$ k tx/s as expected throughput. The estimate aligns with the measured 216 k tx/s, confirming that adaptive read batching reduces CPU overhead in the submission path and improves single-core efficiency.

3.4 Tuning io_uring for the Storage Engine

Utilizing io_uring’s key capabilities enabled us to implement an asynchronous architecture whose performance is determined by CPU cycles spent on I/O processing. This forms the basis for applying low-level features to reduce I/O overheads. However, the effectiveness of io_uring’s features also depends on workload characteristics, as discussed next. The applicability of these features to multi-threaded execution is evaluated separately in Section 3.4.3.

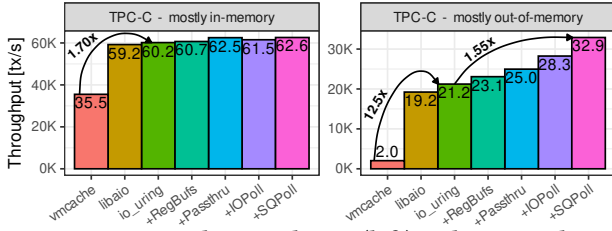


Figure 6: TPC-C with 1 warehouse (left) and 100 warehouses (right) with the default transaction mix. `io_uring` outperforms `libaio`, `vmcache` uses blocking I/O that performs worst if out-of-memory with 100 warehouses (storage-intensive).

3.4.1 Performance Evaluation with YCSB

We cut per-I/O CPU cost by reducing three overheads: data movement (using registered buffers), storage stack (NVMe passthrough), and submission/completion handling (SQPoll & IOPoll).

Registered buffers reduce copies. `io_uring` allows user-space buffers to be *registered* once during initialization, avoiding pinning, DMA setup, and kernel-user copies on every I/O. The kernel then performs DMA directly into user memory, eliminating these overheads. For YCSB, this zero-copy optimization improves throughput by about 11%, reaching 238 k tx/s (Figure 5, `+RegBufs`).

NVMe passthrough skips abstractions. To access NVMe devices directly, `io_uring` provides the `OP_URING_CMD` opcode, which issues native NVMe commands via the kernel to device queues. Compared to kernel-bypass frameworks such as SPDK, these optimizations can be adopted incrementally on top of the regular kernel I/O stack. By bypassing the generic storage stack, passthrough reduces software-layer overhead and per-I/O CPU cost. This yields an additional 20% gain, increasing throughput to 300 k tx/s (Figure 5, `+Passthru`).

IOPoll avoids interrupts. With `IOPOLL`, completion events are polled directly from the NVMe device queue, either by the application or by the kernel `SQPoll` thread (cf. Section 2), replacing interrupt-based signaling. This removes interrupt setup and handling overhead but disables non-pollled I/O, such as sockets, within the same ring. When using filesystems, `IOPOLL` requires explicit support and is typically available only for direct block-device access via `O_DIRECT`. As shown in Figure 5 (`+IOPoll`, right), completion polling provides an additional 21% throughput gain, reaching 376 k tx/s - single-threaded. As we will show later, it also reduces latency for I/O-intensive workloads (cf. Figure 10).

SQPoll eliminates syscalls. In `SQPoll` mode, a dedicated kernel thread continuously polls the submission queue, allowing applications to enqueue I/O requests without calling `io_uring_enter()` for each submission. This dedicates one CPU core to polling but eliminates most syscall and submission overheads. The kernel thread handles I/O completions and places them into the completion queue for later consumption by the application. For our buffer manager (Figure 5, `+SQPoll`), throughput increases by about 32% to 546k tx/s, corresponding to the cost previously spent in syscall and kernel-side processing.

3.4.2 Performance Evaluation with TPC-C

Unlike YCSB, which issues short, independent transactions dominated by random I/O, TPC-C models an OLTP system with interacting transactions and a larger share of in-memory computation. This

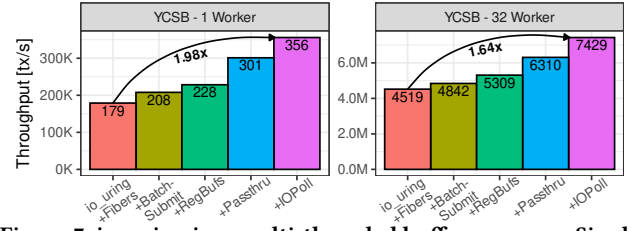


Figure 7: `io_uring` in a multi-threaded buffer manager. Single worker numbers (left) as reference for additional concurrency logic. Optimizations have a similar impact on runtime performance when multiple worker threads are used (right).

workload thus evaluates how `io_uring` optimizations perform in a less I/O-bound, more CPU-intensive setting. As a baseline, we use `vmcache` [32], a state-of-the-art buffer manager. We reuse the asynchronous, batched-read configuration from the YCSB experiments for a fair comparison.

Workload-dependent benefits. Figure 6 shows the results for TPC-C in a mostly in-memory and a mostly out-of-memory configuration. The `io_uring`-based buffer manager consistently outperforms `vmcache`, achieving up to 12.5× higher throughput in the naive configuration. The primary reason is architectural: `vmcache` relies on blocking reads. Enabling advanced `io_uring` features further improves throughput, although the relative gains depend on the workload configuration. The memory-intensive TPC-C workload is largely compute-bound and many reads can be answered from memory which limits the effect of I/O-path optimizations. In this case, `+IOPoll` performs slightly worse than the interrupt-driven baseline because polling wastes CPU cycles when I/O operations are sporadic. `+SQPoll` provides no benefit for the same reason. In the out-of-memory setting, I/O activity rises due to the increased cost of page loads and evictions, making optimizations more impactful.

3.4.3 Multi-Threaded Performance Evaluation

To study whether the `io_uring` optimizations carry over to parallel execution, we extend the buffer manager with a multi-threaded runtime. The runtime uses the same I/O paths as earlier and only introduces additional contention and synchronization handling for worker threads. We use an optimistic-lock-coupling B-tree together with an atomic page table for low-overhead concurrency control. To further reduce synchronization overhead, eviction is organized per shard, and each worker has a dedicated eviction fiber using the same clock-sweep algorithm as in the single-threaded design. Such optimizations are critical to prevent I/O synchronization bottlenecks from masking end-to-end I/O performance impacts.

Relative benefits remain the same. In Figure 7, we report the results of the previous YCSB benchmark when executed using a single worker (performance after multi-threading was added) and 32 worker threads. The results confirm that due to the effective contention and synchronization handling in our multi-threaded buffer manager above the I/O layer, the relative effect of the `io_uring` optimizations remains similar in the multi-threaded setting.

3.5 Take-aways and Summary

When to use `io_uring`. Our buffer manager study shows that `io_uring` yields meaningful gains for I/O-intensive workloads with

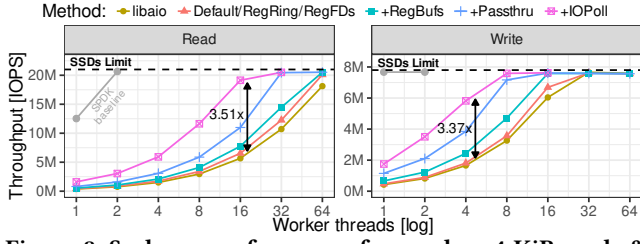


Figure 8: Scale-up performance for random 4 KiB reads & writes between libaio and io_uring with incremental optimizations. io_uring consistently outperforms libaio. Further io_uring optimizations increase throughput by 3.4–3.5×.

many page faults, such as YCSB and TPC-C configurations where a substantial fraction of reads miss the buffer pool and involve SSD page loads and evictions. In compute-heavy (i.e., mostly in-memory) settings, the I/O path contributes little to overall cost, and io_uring optimizations have correspondingly smaller impact.

How to integrate it. A key insight of our study is that io_uring must be integrated as part of an end-to-end architectural design rather than as a drop-in replacement. Using io_uring key features (batching, asynchronous execution) enabled us to shift the bottleneck from device latency to CPU cycles and make the cost of I/O processing explicit, as verified by our model-based analysis.

How to tune it. Once the architecture exposes enough asynchronous I/O, low-level io_uring features can reduce per-operation CPU overhead. However, our study revealed that the tuning benefits depend strongly on the workload: substantial improvements arise when I/O dominates execution time, while CPU-bound or cache-resident workloads gain little.

3.6 Detailed Analysis of io_uring

The results in the previous section showed that realizing performance gains depends on both the system architecture and workload characteristics. In this section, we conduct targeted microbenchmarks to isolate and quantify the effects of individual features for system builders in depth and also study effects of multi-threading. **Batching effects on SSD latency.** In the buffer manager design, we used io_uring’s batching to hide I/O latency for writes and amortize syscall overhead for reads. However, batching can also increase latency variance, which is problematic for workloads requiring predictable response times. To quantify this effect, we vary batch sizes for write requests to a single SSD via io_uring, fixing throughput at 1.5 MIOPS to stay below the device limit and isolate batching behavior. As shown in Table 2, small batches (e.g., size 8) keep latencies mostly below 25 μ s, at the cost of slightly higher syscall frequency. Larger batches reduce submission overhead but cause higher variance; with batch size 128, latency spikes up to 200 μ s occur as bursts from multiple workers can temporarily overload the SSD queue with many outstanding I/Os. Thus, even below I/O saturation, batch size strongly influences latency distribution. For latency-sensitive DBMSs, overly aggressive batching is counterproductive.

Table 2: Impact of batch size on SSD write latency (8 workers).

Batch size:	1	8	32	64	128	256
Latency $\bar{\mu}$ [μ s]	11.51	24.22	60.62	116.40	200.85	317.51
Latency σ [μ s]	± 0.95	± 1.71	± 3.91	± 12.17	± 7.47	± 33.88

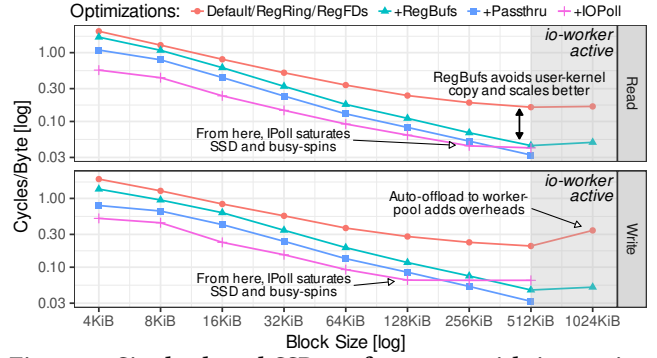


Figure 9: Single thread SSD performance with increasing block sizes. Performance degrades with I/O workers, NVMe-passthrough is only supported until 512KiB.

Multi-threaded performance. To analyze how io_uring behaves under parallelism in isolation, we now increase the number of I/O threads. We use one ring-per-thread for io_uring and also include libaio for comparison. This experiment isolates the scalability of the I/O path and does not use the buffer manager framework from the previous section. As shown in Figure 8, io_uring consistently outperforms libaio for both random reads and writes and exhibits near-linear scalability with the number of threads. At higher core counts, the operating system’s storage stack becomes the dominant bottleneck. The benefit of io_uring optimizations increases with scale: registered buffers (+RegBufs) reduce CPU overhead, while NVMe passthrough (+Passthru) and IOPoll in particular deliver substantial throughput improvements of 3.4–3.5× saturating the SSD array with 18 and 6 cores, respectively.

Increasing block sizes. Larger block sizes for I/O can further amortize CPU costs when the workload allows coarser-grained access. We therefore evaluate how block size affects SSD performance by measuring CPU cycles per byte for reads and writes while varying the I/O block size in a single-threaded setup. Figure 9 shows that larger blocks substantially reduce CPU cost per byte, as syscall and I/O stack overheads are amortized. With sufficiently large requests, a single core saturates the PCIe 5 SSD array, reaching up to 90 GiB/s for reads and 50 GiB/s for writes, close to hardware limits. This point is reached for writes at 128 KiB and reads at 256 KiB with +Passthru and +IOPoll enabled.

Large blocks can backfire. However, exceeding certain thresholds triggers asynchronous worker threads, signaling fallback to slower I/O paths as discussed in Section 2.2. First, if the block size exceeds `max_hw_sectors_kb` (which can be 128 KiB if the IOMMU were enabled), workers are spawned even at low I/O depth, as a single request surpasses the maximum DMA size. Second, with `O_DIRECT`, workers appear once the number of batched requests exceeds `nr_requests` (1023 on bare metal, 127 in our cloud VM), and on some consumer SSDs even without `O_DIRECT`. Third, when block sizes exceed 512 KiB (`max_segments`) asynchronous workers are again used internally for I/O. While large blocks improve efficiency and fully utilize PCIe 5 bandwidth, surpassing these software or hardware limits causes worker fallback, reintroducing latency and CPU overhead in io_uring.

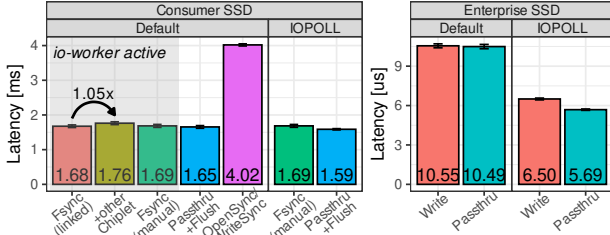


Figure 10: Durable writes with io_uring. Left: Writes and fsync are issued via io_uring or manually linked in the application. Right: Enterprise SSDs do not require explicit fsync.

The durable write problem. Durable writes are essential for database systems, particularly for write-ahead logging and checkpointing, yet remain costly. The standard approach, *fsync*, is blocking in io_uring and thus executed by fallback worker threads. Moreover, *fsync* cannot be issued by rings configured for IOPoll and must be used from a separate ring or as a traditional syscall. These constraints motivate alternatives such as opening files with `O_SYNC`, which delegates durability to the kernel, or using NVMe flushes via passthrough commands. Figure 10 compares these methods on consumer and enterprise SSDs with `O_DIRECT`. Enterprise SSDs with DRAM caches and Power Loss Protection (PLP) achieve microsecond-level latencies, whereas consumer SSDs remain dominated by intrinsic millisecond-scale *fsync* cost, masking worker-thread latencies. Not pinning the *fsync* I/O worker to the local chiplet (+Chiptlet) increases latency by about 5%. `O_SYNC`-based writes perform poorly, more than twice as slow as explicit writes followed by *fsync*. NVMe passthrough with explicit flush commands offers a truly asynchronous durability path but requires raw device access. Linking a write and *fsync* in io_uring offers no improvement over issuing them sequentially in the application. For enterprise SSDs, durability is managed by the device itself, eliminating the need for *fsync*. Passthrough writes with IOPoll reduce latency by about one microsecond, showing the benefit of bypassing the storage stack in latency-sensitive workloads. While passthrough with flush is the most efficient asynchronous option, its lack of filesystem support restricts it to flash-optimized database systems. Implementing durable writes in io_uring therefore requires careful configuration to avoid performance pitfalls.

4 EFFICIENT NETWORK I/O WITH IO_URING

After analyzing storage, we now focus on the networking aspects of io_uring. High-speed interconnects with link rates in the range of 400 Gbit/s are now a commodity in modern data centers and cloud deployments [1, 45]. We study how distributed DBMSs can exploit io_uring in these high-throughput settings and complement the use-case analysis with targeted microbenchmarks.

4.1 Use Case: Distributed Data Shuffle

Shuffle operations are a key building block in any distributed DBMS. We now study a bi-directional, all-to-all data shuffle representative of distributed analytical query execution, as used in parallel joins. **Distributed shuffle overview.** In a distributed shuffle, each node scans local tables and repartitions tuples across nodes according

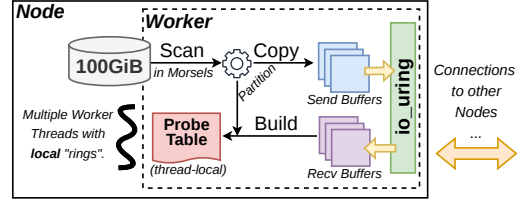


Figure 11: Overview of the shuffle architecture with scan & probe table. Workers use morsel-driven parallelism and handle scanning, probe table building and network I/O.

to a partitioning function. For a distributed hash join, tuples are shuffled by join key. Incoming tuples are inserted into the local probe table, and tuples of the larger relation are later probed there. **Shuffle engine architecture.** We implement a distributed hash join using morsel-based processing [33]. Each worker repeatedly fetches morsels, partitions tuples, and issues send and receive operations while also performing the local hash-table build.

4.2 Workload & System Conditions

Like the buffer manager, the distributed shuffle combines computation and I/O that compete for CPU cycles, but its workload and system conditions differ substantially.

Workload conditions. To study when io_uring yields measurable benefits, we vary tuple size while keeping the total shuffle volume fixed. Smaller tuples cause more probe-table inserts per transferred byte and therefore more random memory accesses, whereas larger tuples reduce insertion frequency and memory pressure.

System conditions. We use the previously described six-node cluster, providing 4.8 Tbit/s of bisection bandwidth ($6 \times 2 \times 400$ Gbit/s), to evaluate shuffle workloads. Since network round-trip latencies are orders of magnitude higher than SSD access latencies, even in traditional engines, it is common to adopt asynchronous strategies such as using dedicated I/O threads [3, 14, 29]. Batching tuples into larger chunks, e.g., 1 MiB, is also an often adopted approach to amortize network overhead and increase throughput [28]. Hence, we adopt an asynchronous design as a baseline without discussing synchronous I/O in the context of the shuffle use case.

4.3 Using io_uring in the Shuffle Engine

Building on these insights and the results of the buffer manager use case in Section 3, we directly utilize io_uring’s asynchronous I/O and batching capabilities for a first shuffle baseline.

Integrating io_uring into a shuffle. The asynchrony of io_uring enables us to avoid the approach of using dedicated I/O threads that use blocking I/O. As recommended with io_uring [8], we adopt a ring-per-thread architecture in which each worker thread possesses a thread-local io_uring ring. Skew is handled at a higher level in the application, not at the I/O submission level. This architecture enables scanning data, while sending and receiving tuples asynchronously via io_uring within a single thread. While both approaches overlap computation and communication, increasing CPU utilization, co-locating computation and I/O within the same thread avoids synchronization overhead with I/O workers and improves cache locality. Unlike in our buffer manager, we directly employ multithreading with the goal to saturate the 400 Gbit/s NICs.

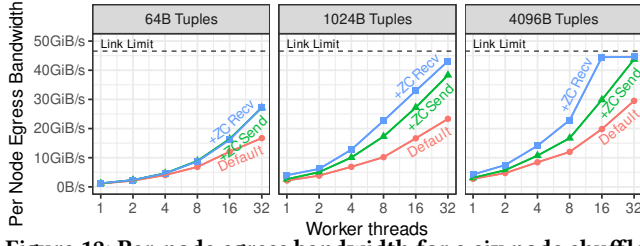


Figure 12: Per-node egress bandwidth for a six node shuffle with different tuple sizes using up to 32 worker threads.

Details of shuffle implementation. To avoid other system bottlenecks and isolate `io_uring` performance impacts, we adopt several state-of-the-art system optimizations in our shuffle implementation. For example, we use batched inserts into the probe table to improve access predictability and allow the prefetcher to issue concurrent loads [10]. We pin workers with their TCP/IP flows in a round-robin manner to CPU *chiplets* [17] to reduce overheads in `io_uring` for networking. Finally, we tune our network stack following industry best practices [31] and use Linux 6.17 for zero-copy recv support.

Basic `io_uring` configuration. `io_uring` comes with multiple network-specific features, such as multi-shot receive (cf. Section 4.6), that can be used for data shuffling. As a baseline, we adopt the same DeferTR and single-issuer setup flags as used in the buffer manager use case to avoid preemptions and internal synchronization (cf. Section 2). For transfers, we use large buffer sizes of 1 MiB, which are more suitable than multi-shot receive or other mechanisms.

Initial performance. Figure 12 shows the per-node egress bandwidth for small and large tuple sizes as the number of worker threads increases. The total bidirectional bandwidth is therefore twice these values. The baseline `io_uring` configuration (red) scales with more workers, mainly due to the morsel-based design, where workers can send and receive data independently. For small tuple sizes (e.g., 64 B), frequent probe-table inserts introduce random memory accesses that stall the CPU and limit network throughput, whereas larger tuples reduce insertion pressure and achieve higher bandwidths. However, even for larger tuples, the system reaches at most 30 GiB/s per node (Figure 12, red), well below the 400 Gbit/s link rate, indicating that the workload is not I/O-bound.

4.4 Tuning `io_uring` for Network I/O

As with storage, `io_uring` also provides multiple network optimizations to reduce per-I/O overhead. In the following, we discuss our findings on how to best use them for efficient network shuffling.

Reducing memory pressure. At high network speeds, copies between user-space and kernel buffers become costly because even a single extra copy consumes substantial memory bandwidth. To reduce this overhead, `io_uring` supports zero-copy send and receive, transmitting data directly from or into pinned user-space buffers.

Zero-copy I/O reduces memory load. As shown in Figure 12, enabling zero-copy send (green) and zero-copy receive (blue) yields visible throughput improvements across different tuple sizes compared to the default setting. For 4 KiB tuples, link bandwidth is saturated with only 16 workers per node when both zero-copy paths are active, achieving full bidirectional 400 Gbit/s utilization with modest CPU usage. However, for 64B tuples, zero-copy receive

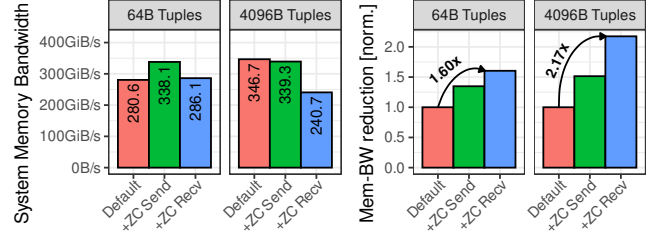


Figure 13: Memory bandwidth for the data shuffle in Fig. 12 (32 Workers). Left: absolute bandwidth; right: Bandwidth reduction normalized by achieved network throughput.

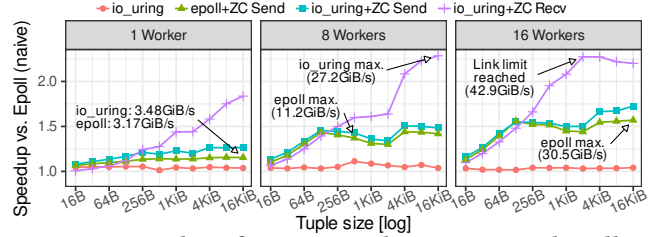


Figure 14: Speedup of `io_uring` and zero-copy send `epoll` vs. plain `epoll` for data shuffling across six nodes and different tuple sizes. Zero-copy receive is only available with `io_uring`.

does not provide additional benefits beyond zero-copy send. We suspect that contention between NIC traffic and CPU-driven random memory accesses diminishes the potential gains, but a deeper analysis of zero-copy receive is left for future work.

Analyzing memory bandwidth. To understand why shuffle performance is limited without zero-copy operations, we analyze system memory bandwidth during the scale-up experiment described above. Figure 13 shows measurements for both the default and zero-copy configurations. On the left, we present the system memory bandwidth, calculated as the sum of read and write traffic from hardware performance counters. For both workloads, peak system bandwidth approaches 400 GiB/s. Zero-copy configurations typically show equal or higher absolute memory bandwidth, primarily due to their higher network throughput, which increases overall memory traffic. To account for this effect, we normalize the measured bandwidth by the achieved network throughput. The normalized plot (cf. Figure 13, right) shows that using zero-copy for send and receive reduces effective memory bandwidth by about half, as expected, since data copies are eliminated in both directions.

End-to-end comparison with `epoll`. Similar to the buffer manager, we compare our optimized `io_uring`-enabled implementation to state-of-the-art approaches. For the shuffle, we use `epoll` as a baseline, a readiness-based I/O interface commonly used in existing systems. Figure 14 shows the speedup of different `io_uring` variants and an `epoll`-based zero-copy send implementation against a naive `epoll` baseline without optimizations. We vary tuple sizes (smaller tuples result in more random inserts into the probe table per byte transferred) and scale up to sixteen workers to avoid link saturation and ensure a fair comparison. Without zero-copy, `epoll` is only marginally slower than `io_uring` despite issuing more system calls. This behavior stems from both implementations transferring tuples

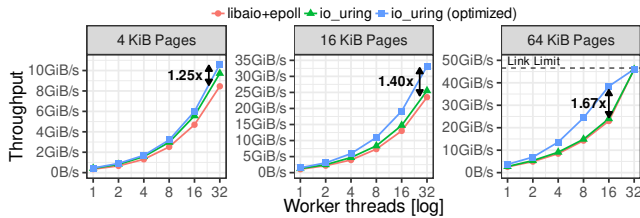


Figure 15: Remote table scan combining storage and network I/O. A client requests pages of different sizes over 32 concurrent connections per worker from a server that reads them from SSD and returns them over the network.

in large 1 MiB chunks, which amortizes syscall and I/O-path overhead. With zero-copy send, `io_uring` achieves substantially better performance, with the gap widening as the number of workers increases. Unlike `epoll`, `io_uring` also supports zero-copy receive, further improving performance and providing up to a 2.5× speedup. **Unified storage and network I/O.** To evaluate the practical benefit of `io_uring`’s unified asynchronous interface, we measure a remote table scan in which a server reads data pages from SSD and sends them to a client over the network. With `io_uring`, storage and network operations are submitted and completed through the same ring, whereas `epoll+libaio` must coordinate separate storage and network subsystems and their event handling. The results in Figure 15 show that `io_uring` (green+blue) consistently outperforms the split `libaio+epoll` design (red), with the gap widening further once registered buffers and zero-copy are enabled, while for larger page sizes the difference narrows because costs are amortized.

4.5 Take-aways and Summary

When to use `io_uring`. Our shuffle use case shows that `io_uring` provides benefits in high-throughput settings where memory bandwidth becomes a bottleneck. With large tuples, when probe table inserts are rare, and workers mainly stream tuples and perform simple partitioning, zero-copy networking enables us to saturate 400 Gbit/s links with relatively few cores. For smaller tuple sizes, random memory accesses during hash-table inserts dominate and limit throughput; network-path optimizations have a less significant impact but remain important for achieving optimal bandwidth.

How to integrate it. Treating `io_uring` as a drop-in replacement in a traditional I/O-worker design is inadequate. It requires a ring-per-thread design that overlaps computation and I/O within the same thread. Combined with careful placement of workers across chiplets and an optimized networking stack, this architecture keeps cores busy and exposes concurrent I/O for `io_uring` to be effective.

How to tune it. Once the engine operates asynchronously with large batched transfers, tuning `io_uring` can reduce data movement and CPU overhead per I/O. Registered buffers and zero-copy send/receive eliminate kernel-user copies, reducing memory bandwidth consumption per unit of network throughput by $\approx 2\times$.

4.6 Detailed Analysis of `io_uring`

In this section, we focus on important aspects of `io_uring` for tuning network interfaces of database systems, which we could not cover before through targeted microbenchmarks.

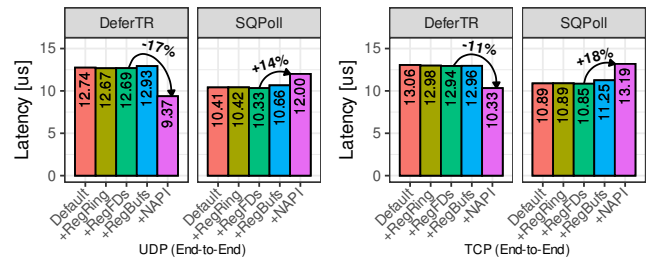


Figure 16: UDP and TCP latency ping-pong with 8-byte messages and incrementally enabled optimizations. NAPI polling reduces latency for DeferTR, but increases it with SQPoll.

Reducing latency with `io_uring`. Network latency is crucial for latency-sensitive protocols, such as transaction coordination or replication, where microsecond-level delays accumulate at scale. We measure one-way and round-trip latencies for TCP and UDP using 8-byte messages to capture the minimum cost of message delivery. Both modes - deferred taskrun (DeferTR) and SQPoll - are evaluated, along with optimizations such as registered file descriptors, registered buffers, and NAPI (the networking counterpart of IOPoll). As shown in Figure 16, SQPoll achieves lower latency than DeferTR, but the advantage disappears once NAPI is enabled. Without NAPI, SQPoll mainly helps by removing submission syscalls. With NAPI, the remaining handoff cost of SQPoll can dominate for very small messages. DeferTR with NAPI yields the best latency, outperforming SQPoll, while registered buffers have a negligible impact for small messages and increase latency slightly. When the NIC queue is pinned to a remote chiplet, cross-chiplet interrupt handling increases latency by 14 % for UDP and 21 % for TCP. With NAPI enabled, however, remote queues cause only a negligible increase of less than 1 %. For reference, a DPDK-based implementation reaches 7 μ s, providing a lower bound for userspace networking.

Send path optimizations. As shown in Figure 16, registered buffers have little effect for very small messages, in contrast to their positive impact in the storage setting (Section 3.4.1). To better understand which optimizations are most effective for different database workloads, we vary the transfer size and report the effective cycle cost per transmitted byte. Figure 17 (left) shows a clear threshold around 1 KiB: below this size, zero-copy send performs worse than plain `io_uring` due to buffer-management overheads, whereas for larger messages registered buffers amortize this cost and consistently reduce per-byte CPU time. Registering rings and file descriptors yields only marginal improvements but introduces no observable drawbacks. Overall, zero-copy with registered buffers is the most efficient configuration for large messages, achieving up to 3.5× fewer cycles per transmitted byte than default `io_uring`.

Receive path optimizations. We observe analogous thresholds for the receive path (Figure 17, right). Multishot receive operations repeatedly generate completions from a single submission and are most efficient for workloads with small messages. Once message sizes exceed roughly 1 KiB, zero-copy receive becomes more efficient, and for very large messages (e.g., 13 KiB and above) even the normal single-shot receive path outperforms multishot due to reduced per-message overheads. `io_uring` can also draw receive buffers from a kernel-managed pool (RingBufs), but these perform worse than user-supplied buffers and are only useful in multishot

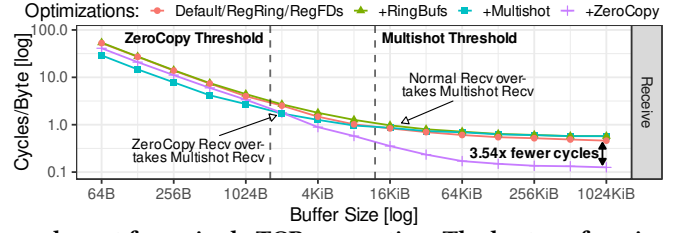
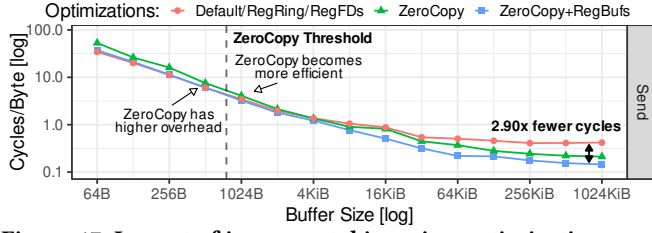


Figure 17: Impact of incremental `io_uring` optimizations on the cycle cost for a single TCP connection. The best-performing configuration depends on the shown thresholds. Registered file descriptors offer minimal benefit and are therefore omitted.

scenarios. Exact thresholds vary with capabilities and available offloads of the NIC, but the overall pattern remains consistent.

Optimizing kernel execution for sockets. Recall how `io_uring` executes `task_work` inside the kernel. By default, it first attempts to complete an I/O operation inline in non-blocking mode and falls back to internal polling only if the call returns `-EAGAIN` (see Figure 3). For socket operations, this speculative attempt can be wasteful when the application already knows the socket is empty (for receive) or full (for send), causing unnecessary kernel work. Such cases commonly occur in RPC-style communication, where the response is expected only after the request. To handle this efficiently, `io_uring` provides the `RECVSEND_POLL_FIRST` flag, which skips the speculative attempt and directly uses the poll set. Using `PollFirst` reduces the number of instructions executed and kernel work, resulting in up to $1.5\times$ reduction in CPU cycles spent.

5 INSIGHTS FOR SYSTEM BUILDERS

In our study, we focused on three research questions: *when to use `io_uring`*, *how to integrate it*, and *how to tune it*. We summarize these insights as actionable guidelines for engineers, and then validate their practicality by applying them to enhance PostgreSQL’s.

5.1 Guidelines

Based on our system case studies, we draw four practical guidelines for using `io_uring` effectively in database systems:

- (1) **Determine if I/O is a system bottleneck.** When I/O accounts for only a small fraction of execution time, as in CPU-bound or cache-resident workloads, potential `io_uring` gains are limited. Our use cases in Section 3 and Section 4 show that `io_uring` is most effective when it reduces or amortizes the CPU cost of I/O operations, or when it lowers the memory bandwidth consumption. As demonstrated in the buffer manager case study, simple latency or cycle models help to model such bottlenecks.
- (2) **Align the architecture with `io_uring` capabilities.** `io_uring` enables asynchronous execution, system call batching, and a unified interface for storage and network I/O. Our buffer manager (Section 3.1) shows that architectural changes, such as overlapping I/O and computation via asynchronous execution or amortizing per-I/O cost through batching, can yield large improvements. The network shuffle (Section 4.1) shows how applications can use the *ring-per-thread* architecture to scale beyond a single core.
- (3) **Choose and tune the execution mode deliberately.** The recommended `io_uring` configuration uses `DeferTR` with *single-issuer* for predictable task execution and controlled completion reaping (Section 2). `SQPoll` can improve performance when dedicating a

polling core is amortized, and latency or IOPS targets justify the additional CPU cost. Falling back to `io_workers` should be avoided by ensuring that all operations, including `fsync` and large I/Os, can execute fully asynchronously (cf. Figure 9). Tuning the execution mode to the underlying hardware helps avoid costly effects, such as cross-chiplet traffic or non-local interrupt handling (Section 4.3).

(4) **Use `io_uring` optimizations.** `io_uring` offers a range of general, storage-specific, and networking-specific optimizations for I/O-intensive systems. Carefully selecting these optimizations can reduce the per-I/O cycle cost. Some optimizations, such as registered FDs or fixed buffers for 4 KiB page-aligned storage I/O, do not negatively impact performance (see Section 3.4.1). However, mechanisms like zero-copy or multishot receive are only effective when payloads exceed device-specific thresholds (cf. Figure 17). Other optimizations, such as NVMe passthrough or `IOPoll`, apply only when no filesystem or a compatible filesystem is used.

5.2 Optimizing PostgreSQL using Guidelines

PostgreSQL recently added `io_uring` support in version 18 for asynchronous data and WAL access [19]. We use this integration to show how the guidelines apply in a production-grade engine despite PostgreSQL’s architectural constraints to improve performance of a cold table scan from disk.

GL (1): Determine if I/O is a system bottleneck. Even without tuning `io_uring` parameters or enabling additional features, PostgreSQL’s new `io_uring` backend achieves up to $3\times$ higher performance in I/O-intensive workloads than the previous synchronous design based on blocking calls and OS readahead [19]. This confirms that I/O is a dominant bottleneck and provides the baseline for the improvements in Figure 18, measured with 1–8 backend workers scanning a 32 GiB cold table via direct I/O.

GL (2): Align the architecture with `io_uring` capabilities. PostgreSQL already partially aligns with guideline (2) by overlapping computation and I/O through multiple asynchronous reads and writes. However, its multi-process design allows backends to wait on I/O submitted through rings owned by other processes, so rings are not used by a single issuer and `DeferTR` cannot be applied directly. A more `io_uring`-friendly design would use one ring per thread with exclusive ownership and no cross-process sharing.

GL (3): Choose and tune the execution mode deliberately. Given PostgreSQL’s architecture, `DeferTR` cannot be used without substantial refactoring. We therefore configure `io_uring` with `CoopTR` as the next-best alternative. `CoopTR` avoids kernel-driven `task_work` preemptions while remaining compatible with PostgreSQL’s process model. Still following guideline (3), we extend

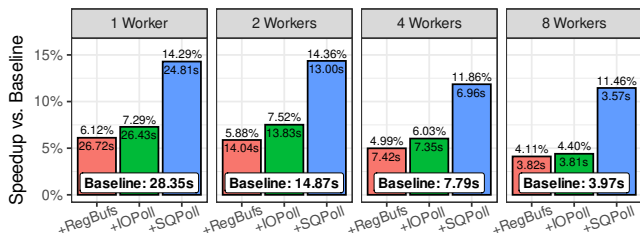


Figure 18: PostgreSQL speedup from io_uring optimizations. Sharing the SQPoll kernel thread between rings has negligible performance impact. Improvements remain limited by the filesystem and PostgreSQL’s multi-process architecture.

the backend to support SQPoll by enabling the respective setup flag during ring creation and allowing multiple backend processes to share a single SQPoll kernel thread. For WAL durability, PostgreSQL invokes `fsync()` directly (or via `O_DATASYNC`) rather than through `io_uring`, consistent with guideline (3). This avoids spawning `io_workers` for blocking `fsync()` calls and keeps the critical path fully asynchronous.

GL (4): Use io_uring optimizations. Because PostgreSQL relies on filesystems for data storage, low-level optimizations such as NVMe passthrough are unavailable, and `IOPoll` depends on filesystem support. We apply guideline (4) by enabling `io_uring` optimizations that fit PostgreSQL’s access patterns and architectural constraints. We register the entire buffer pool as fixed buffers, eliminating PostgreSQL’s heuristic of enabling `IO_ASYNC` after four outstanding I/Os. This matches our earlier findings for 8 KiB pages (Figure 9). In Figure 18, fixed buffers alone provide a 4–6% improvement. Because we use `ext4`, we additionally enable `IOPoll`, improving performance up to 7.5% over the baseline. Combined with `SQPoll`, total throughput improves by 11–15% over upstream PostgreSQL, despite the architectural constraints discussed above.

Summary. Applying the guidelines yields speedups even in a mature DBMS such as PostgreSQL, despite architectural and filesystem constraints that limit the applicability of several `io_uring` features.

6 RELATED WORK

Although `io_uring` is a relatively recent addition to the Linux kernel, it has already been studied in various contexts, primarily focusing on storage I/O and its impact on data-intensive systems [42, 43].

Storage interfaces and NVMe systems. Initial systematic comparisons of `io_uring` with established interfaces such as `libaio` and `SPDK` were conducted using the `fio` benchmark to evaluate typical storage workloads [13]. Ren and Trivedi extended this analysis for Intel Optane SSDs, characterizing microarchitectural behavior, instruction-level overheads, and Linux block I/O scheduler performance [44]. In the context of database systems, Haas et al. explored `io_uring` and other asynchronous I/O mechanisms for NVMe SSD arrays. They identified improvements in throughput and latency associated with various `io_uring` features [21–23].

Application-level and system integrations. Several works integrated `io_uring` into production systems to evaluate application-specific benefits. Chen et al. applied `io_uring` to Redis [12], reporting significant reductions in overhead for medium and large payloads. Durner et al. used `io_uring` to accelerate cloud object storage access,

achieving lower latency and higher throughput [15]. However, systematic analyses of network-specific aspects and end-to-end effects on distributed systems remain absent.

Security and advanced feature analyses. From a security perspective, He et al. proposed RingGuard [25], an eBPF-based framework that monitors and restricts `io_uring` operations to prevent kernel-level vulnerabilities. It extends eBPF with new `io_uring`-specific hooks and verifier logic, enforcing safety policies at runtime while maintaining low overhead [25]. The most comprehensive exploration of `io_uring` features to date is the work by Ingimarsson, who integrated basic functionality into RocksDB [26]. However, detailed evaluations of advanced `io_uring` features, such as registered buffers for zero-copy operations and linked requests to minimize system calls, are still lacking.

Async I/O in Windows. Windows’ `IORing` API exposes shared submission and completion queues and supports batched submission, but the currently documented interface primarily targets file-oriented operations [36, 37]. Network I/O instead relies on the older *Overlapped* I/O model, which supports both storage and networking but requires one system call per submitted request [38]. Thus, unlike `io_uring`, current Windows interfaces do not provide a unified queue-based abstraction for both storage and network I/O.

7 CONCLUSION AND OUTLOOK

The modern Linux `io_uring` interface provides powerful mechanisms for asynchronous I/O, but achieving measurable gains requires more than simply enabling its features. Our case studies show that performance depends on understanding system bottlenecks, integrating `io_uring` into the overall design, exploiting batching and asynchronous execution, and applying targeted optimizations to reduce CPU cost per I/O. Focused microbenchmarks further highlight trade-offs such as zero-copy buffer-size effects and the importance of aligning polling strategies with hardware and workload characteristics. From this analysis, we derived practical guidelines for I/O-intensive systems and validated them through an integration into PostgreSQL, improving table-scan throughput by 11–15% over its baseline `io_uring` implementation. As `io_uring` continues to evolve, adopting it today makes systems future-ready by enabling them to benefit from new capabilities such as zero-copy receive without architectural redesign.

ACKNOWLEDGMENTS

This research was partially funded by the LOEWE Spitzenprofessor of the state of Hesse (III 5-519/05.00.003-(0005)), and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy (EXC3057/1 “Reasonable Artificial Intelligence”, Project No. 533677015).

 Funded/Co-funded by the European Union (ERC, CODAC, 101041375). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

We thank Jens Axboe and the Linux kernel community for their valuable feedback and support regarding `io_uring`. We also thank DFKI Darmstadt and hessian.AI for their support.

REFERENCES

- [1] 2025. Amazon EC2 Instances. <https://aws.amazon.com/ec2/instance-types/>. Accessed: 2025-11-27.
- [2] 2025. Boost.fiber framework. <https://github.com/boostorg/fiber>. Accessed: 2025-08-22.
- [3] 2025. *MySQL 8.4 Reference Manual*. <https://dev.mysql.com/doc/refman/8.4/en/innodb-linux-native-aio.html> Accessed: 2025-08-18.
- [4] Jens Axboe. 2019. Efficient I/O with io_uring. https://kernel.dk/io_uring.pdf. Accessed: 2025-10-17.
- [5] Jens Axboe. 2020. *Re: io_uring is slower than epoll (issue #189 comment)*. GitHub issue comment on the liburing repository. Accessed: 2025-08-11.
- [6] Jens Axboe. 2021. *Re: unexpected high count of io_worker threads by using the IOSQE_ASYNC flag (issue #349 comment)*. GitHub issue comment on the liburing repository. Accessed: 2025-08-11.
- [7] Jens Axboe. 2024. *Re: EAGAINs impacting the performance of io_uring (issue #1175 comment)*. GitHub issue comment on the liburing repository. Accessed: 2025-08-11.
- [8] Jens Axboe. 2025. *io_uring and networking in 2023*. https://github.com/axboe/liburing/wiki/io_uring-and-networking-in-2023#task-work Accessed: 2025-10-22.
- [9] Jens Axboe. 2025. io_uring library liburing. <https://github.com/axboe/liburing/>. Accessed: 2025-10-17.
- [10] Altan Birler, Tobias Schmidt, Philipp Fent, and Thomas Neumann. 2024. Simple, Efficient, and Robust Hash Tables for Join Processing. In *Proceedings of the 20th International Workshop on Data Management on New Hardware, DaMoN 2024, Santiago, Chile, 10 June 2024*, Carsten Binnig and Nesime Tatbul (Eds.). ACM, 4:1–4:9. <https://doi.org/10.1145/3662010.3663442>
- [11] Matthew Butrovich, Karthik Ramanathan, John Rollinson, Wan Shen Lim, William Zhang, Justine Sherry, and Andrew Pavlo. 2023. Tigger: A Database Proxy That Bounces With User-Bypass. *Proc. VLDB Endow.* 16, 11 (2023), 3335–3348. <https://doi.org/10.14778/3611479.3611530>
- [12] Le-Gao Chen, Yanzhi Li, Tipporn Laohakangvalvit, and Midori Sugaya. 2024. Asynchronous I/O Persistence for In-Memory Database Servers: Leveraging io_uring to Optimize Redis Persistence. In *CLOUD Computing - CLOUD 2024 - 17th International Conference, Held as Part of the Services Conference Federation, SCF 2024, Bangkok, Thailand, November 16-19, 2024, Proceedings (Lecture Notes in Computer Science)*, Yang Wang and Liang-Jie Zhang (Eds.), Vol. 15423. Springer, 11–20. https://doi.org/10.1007/978-3-031-77153-8_2
- [13] Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. 2022. Understanding modern storage APIs: a systematic study of libaio, SPDK, and io_uring. In *SYSTOR '22: The 15th ACM International Systems and Storage Conference, Haifa, Israel, June 13 - 15, 2022*, Michal Malka, Hillel Kolodner, Frank Bellosa, and Moshe Gabel (Eds.). ACM, 120–127. <https://doi.org/10.1145/3534056.3534945>
- [14] Aleksandar Dragojevic, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. 2014. FaRM: Fast Remote Memory. In *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2014, Seattle, WA, USA, April 2-4, 2014*, Ratul Mahajan and Ion Stoica (Eds.). USENIX Association, 401–414. <https://www.usenix.org/conference/nsdi14/technical-sessions/dragojevic%28C4%87>
- [15] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proc. VLDB Endow.* 16, 11 (2023), 2769–2782. <https://doi.org/10.14778/3611479.3611486>
- [16] Wolfgang Effelsberg and Theo Härder. 1984. Principles of Database Buffer Management. *ACM Trans. Database Syst.* 9, 4 (1984), 560–595. <https://doi.org/10.1145/1994.2022>
- [17] Alessandro Fogli, Bo Zhao, Peter R. Pietzuch, Maximilian Bandle, and Jana Giceva. 2024. OLAP on Modern Chiplet-Based Processors. *Proc. VLDB Endow.* 17, 11 (2024), 3428–3441. <https://doi.org/10.14778/3681954.3682011>
- [18] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Ñigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. 2024. Making Kernel Bypass Practical for the Cloud with Junction. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, Laurent Vanbever and Irene Zhang (Eds.). USENIX Association, 55–73. <https://www.usenix.org/conference/nsdi24/presentation/fried>
- [19] PostgreSQL Global Development Group. 2025. *PostgreSQL 18 Released*. <https://www.postgresql.org/about/news/postgresql-18-released-3142/>
- [20] The PostgreSQL Global Development Group. 2005. PostgreSQL 8.4.22 Documentation - Appendix E. Release Notes. <https://www.postgresql.org/docs/8.4/release-8-1.html>. Accessed: 2025-10-10.
- [21] Gabriel Haas, Adnan Alhomssi, and Viktor Leis. 2025. Managing Very Large Datasets on Directly Attached NVMe Arrays. In *Scalable Data Management for Future Hardware*, Kai-Uwe Sattler, Alfons Kemper, Thomas Neumann, and Jens Teubner (Eds.). Springer Nature Switzerland, Cham, 223–240. https://doi.org/10.1007/978-3-031-74097-8_9
- [22] Gabriel Haas, Michael Haubenschield, and Viktor Leis. 2020. Exploiting Directly-Attached NVMe Arrays in DBMS. In *10th Conference on Innovative Data Systems Research, CIDR 2020, Amsterdam, The Netherlands, January 12-15, 2020, Online Proceedings*. www.cidrdb.org. <http://cidrdb.org/cidr2020/papers/p16-haas-cidr20.pdf>
- [23] Gabriel Haas and Viktor Leis. 2023. What Modern NVMe Storage Can Do, And How To Exploit It: High-Performance I/O for High-Performance Storage Engines. *Proc. VLDB Endow.* 16, 9 (2023), 2090–2102. <https://doi.org/10.14778/3598581.3598584>
- [24] Haochen He, Erci Xu, Shanshan Li, Zhouyang Jia, Si Zheng, Yue Yu, Jun Ma, and Xiangke Liao. 2023. When Database Meets New Storage Devices: Understanding and Exposing Performance Mismatches via Configurations. *Proc. VLDB Endow.* 16, 7 (2023), 1712–1725. <https://doi.org/10.14778/3587136.3587145>
- [25] Wanning He, Hongyi Lu, Fengwei Zhang, and Shuai Wang. 2023. RingGuard: Guard io_uring with eBPF. In *Proceedings of the 1st Workshop on eBPF and Kernel Extensions, eBPF 2023, New York, NY, USA, 10 September 2023*. ACM, 56–62. <https://doi.org/10.1145/3609021.3609304>
- [26] Brynjar Ingimarsson. 2024. *Exploring the Performance of the Io_uring Kernel I/O Interface*. Master's thesis. Universiteit van Amsterdam, Amsterdam.
- [27] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, and Carsten Binnig. 2025. A Wake-Up Call for Kernel-Bypass on Modern Hardware. In *Proceedings of the 21st International Workshop on Data Management on New Hardware, DaMoN 2025, Berlin, Germany, June 22-27, 2025*. ACM, 14:1–14:5. <https://doi.org/10.1145/3736227.3736235>
- [28] Theo Jepsen, Alberto Lerner, Fernando Pedone, Robert Soul é, and Philippe Cudré-Mauroux. 2021. In-Network Support for Transaction Triaging. *Proc. VLDB Endow.* 14, 9 (2021), 1626–1639. <https://doi.org/10.14778/3461535.3461551>
- [29] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2019. Datacenter RPCs can be General and Fast. In *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, Jay R. Lorch and Minlan Yu (Eds.). USENIX Association, 1–16. <https://www.usenix.org/conference/nsdi19/presentation/kalia>
- [30] Daehyeok Kim, Amir Saman Memaripour, Anirudh Badam, Yibo Zhu, Hongqiang Harry Liu, Jitu Padhye, Shachar Raindel, Steven Swanson, Vyas Sekar, and Srinivasan Seshan. 2018. Hyperloop: group-based NIC-offloading to accelerate replicated transactions in multi-tenant storage systems. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM 2018, Budapest, Hungary, August 20-25, 2018*, Sergey Gorinsky and János Tapolcai (Eds.). ACM, 297–312. <https://doi.org/10.1145/3230543.3230572>
- [31] Microsoft Learn. 2024. Optimize network throughput for Azure virtual machines. <https://learn.microsoft.com/en-us/azure/virtual-network/virtual-network-optimize-network-bandwidth#achieving-consistent-transfer-speeds-in-linux-vms-in-azure>. Accessed: 2025-10-17.
- [32] Viktor Leis, Adnan Alhomssi, Tobias Ziegler, Yannick Loeck, and Christian Dietrich. 2023. Virtual-Memory Assisted Buffer Management. *Proc. ACM Manag. Data* 1, 1 (2023), 7:1–7:25. <https://doi.org/10.1145/3588687>
- [33] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22-27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, 743–754. <https://doi.org/10.1145/2588555.2610507>
- [34] Viktor Leis and Christian Dietrich. 2024. Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware. *Proc. VLDB Endow.* 17, 8 (2024), 2115–2122. <https://doi.org/10.14778/3659437.3659462>
- [35] Feng Li, Sudipto Das, Manoj Syamala, and Vivek R. Narasayya. 2016. Accelerating Relational Databases by Leveraging Remote Memory and RDMA. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). ACM, 355–370. <https://doi.org/10.1145/2882903.2882949>
- [36] Microsoft. 2026. I/O Rings. <https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-rings>. Microsoft Learn, accessed 2026-03-17.
- [37] Microsoft. 2026. *io_uring.h* header - Win32 apps. <https://learn.microsoft.com/en-us/windows/win32/api/ioringapi/>. Microsoft Learn, accessed 2026-03-17.
- [38] Microsoft. 2026. Synchronous and Asynchronous I/O. <https://learn.microsoft.com/en-us/windows/win32/fileio/synchronous-and-asynchronous-i-o>. Microsoft Learn, accessed 2026-03-17.
- [39] Vivek R. Narasayya, Ishai Menache, Mohit Singh, Feng Li, Manoj Syamala, and Surajit Chaudhuri. 2015. Sharing Buffer Pool Memory in Multi-Tenant Relational Database-as-a-Service. *Proc. VLDB Endow.* 8, 7 (2015), 726–737. <https://doi.org/10.14778/2752939.2752942>
- [40] Lam-Duy Nguyen, Adnan Alhomssi, Tobias Ziegler, and Viktor Leis. 2025. Moving on From Group Commit: Autonomous Commit Enables High Throughput and Low Latency on NVMe SSDs. *Proc. ACM Manag. Data* 3, 3 (2025), 191:1–191:24. <https://doi.org/10.1145/3725328>
- [41] NVIDIA. 2021. *NVIDIA ConnectX-7 Datasheet*. <https://www.nvidia.com/content/dam/en-zz/Solutions/networking/infiniband-adapters/infiniband-connectx7-data-sheet.pdf> Accessed: 2025-03-18.

- [42] Constantin Pestka and Marcus Paradies. 2025. Tutorial: Dreaming of Syscall-less I/O with `io_uring` - Some Assembly Required, Feaver Dreams and Nightmares included. In *Datenbanksysteme für Business, Technologie und Web (BTW 2025) Workshopband, Bamberg, Germany, March 3-7, 2025 (LNI)*, Carsten Binnig, Andreas Henrich, Daniela Nicklas, Maximilian E. Schüle, and Klaus Meyer-Wegener (Eds.), Vol. P-363. Gesellschaft für Informatik e.V., 385–388. <https://doi.org/10.18420/BTW2025-143>
- [43] Constantin Pestka, Marcus Paradies, and Matthias Pohl. 2024. Asynchronous I/O - With Great Power Comes Great Responsibility. *CoRR* abs/2411.16254 (2024). <https://doi.org/10.48550/ARXIV.2411.16254> arXiv:2411.16254
- [44] Zebin Ren and Animesh Trivedi. 2023. Performance Characterization of Modern Storage Stacks: POSIX I/O, `libaio`, `SPDK`, and `io_uring`. In *Proceedings of the 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems, CHEOPS 2023, Rome, Italy, 8 May 2023*, Jean-Thomas Acquaviva, Shadi Ibrahim, and Suren Byna (Eds.). ACM, 35–45. <https://doi.org/10.1145/3578353.3589545>
- [45] Bowen Wu, Wei Cui, Carlo Curino, Matteo Interlandi, and Rathijit Sen. 2025. Terabyte-Scale Analytics in the Blink of an Eye. *Proc. VLDB Endow.* 19, 2 (2025), 141–155. <https://www.vldb.org/pvldb/vol19/p141-sen.pdf>
- [46] Xinjing Zhou, Viktor Leis, Xiangyao Yu, and Michael Stonebraker. 2025. OLTP Through the Looking Glass 16 Years Later: Communication is the New Bottleneck. In *15th Conference on Innovative Data Systems Research, CIDR 2025, Amsterdam, The Netherlands, January 19-22, 2025*. www.cidrdb.org. <https://vldb.org/cidrdb/2025/oltp-through-the-looking-glass-16-years-later-communication-is-the-new-bottleneck.html>